Image Classifier For Fruits

Internship Project Report

Codec Technologies Internship

1. Introduction

Fruit identification is a common task in agriculture, retail, and food processing industries. Manually sorting and identifying fruits is time-consuming and prone to error. This project explores how a Convolutional Neural Network (CNN), a deep learning technique well-suited for image data, can automatically classify images of fruits into their respective categories (for example, apple, banana, orange, mango, grapes).

The project was carried out as part of a Machine Learning / AI internship, with the goal of gaining hands-on experience in image classification, deep learning model building, and deployment basics.

1.1 Objective

- To build an image classification model that can identify different types of fruits from images.
- To understand and apply Convolutional Neural Networks (CNN) using Python.
- To learn the complete machine learning pipeline: data collection, preprocessing, training, evaluation, and prediction.
- To evaluate model performance using accuracy and other relevant metrics.

1.2 Scope

The classifier is trained to recognize a limited set of common fruit classes from static images. It is intended as a learning project to demonstrate core image classification concepts rather than a production-grade industrial system.

2. Background

Image classification is a core computer vision task where a model assigns a label to an input image. Traditional approaches relied on hand-crafted features (color histograms, shape descriptors), but modern approaches use deep learning, particularly CNNs, which automatically learn relevant features directly from raw pixel data.

A CNN typically consists of convolutional layers (to detect patterns like edges and textures), pooling layers (to reduce dimensionality), and fully connected layers (to perform final classification). This architecture makes CNNs highly effective for image-based tasks.

3. Tools and Technologies Used

Category	Tools / Technology
Programming Language	Python 3
Deep Learning Framework	TensorFlow / Keras (or PyTorch)
Data Handling	NumPy, Pandas

Image Processing	OpenCV, PIL (Pillow)
Visualization	Matplotlib, Seaborn
Development Environment	Jupyter Notebook / Google Colab
Dataset	Fruits-360 dataset (Kaggle) or a custom collected dataset

4. Dataset Description

The dataset used consists of labeled images of different fruit categories, organized into folders by class (one folder per fruit type). A commonly used public dataset for this purpose is the Fruits-360 dataset, which contains thousands of images across 100+ fruit and vegetable classes.

4.1 Dataset Details (Example)

- Number of classes used: 5 (e.g., Apple, Banana, Orange, Mango, Grapes)
- Total images: ~3,000–5,000 (varies by classes selected)
- Image size: Resized to 100x100 or 224x224 pixels
- Split: 80% training, 10% validation, 10% testing

5. Methodology

5.1 Data Preprocessing

1. Loaded images from the dataset folders using image data generators.
2. Resized all images to a fixed dimension for consistency.
3. Normalized pixel values (scaled from 0–255 to 0–1) for faster and stable training.
4. Applied data augmentation (rotation, flipping, zoom) to increase data variety and reduce overfitting.
5. Split the dataset into training, validation, and test sets.

5.2 Model Architecture

A simple CNN architecture was built using Keras Sequential API:

```
Conv2D(32 filters, 3x3) -> ReLU -> MaxPooling2D
Conv2D(64 filters, 3x3) -> ReLU -> MaxPooling2D
Conv2D(128 filters, 3x3) -> ReLU -> MaxPooling2D
Flatten
Dense(128) -> ReLU -> Dropout(0.5)
Dense(num_classes) -> Softmax
```

Optionally, transfer learning with a pre-trained model (e.g., MobileNetV2) can be used to improve accuracy with less training data and time.

5.3 Model Training

- Loss function: Categorical Cross-Entropy
- Optimizer: Adam
- Batch size: 32
- Epochs: 15–25 (with early stopping based on validation loss)
- Evaluation metric: Accuracy

5.4 Sample Code Snippet

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(100,100,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(num_classes, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(train_data, validation_data=val_data, epochs=20)
```

6. Results and Evaluation

The trained model was evaluated on the unseen test dataset. Below is a sample summary of results (replace with your actual training results):

Metric	Value
Training Accuracy	~97%
Validation Accuracy	~94%
Test Accuracy	~92–95%
Loss	Low and stable after ~15 epochs

A confusion matrix and accuracy/loss curves were plotted to visualize model performance across epochs and identify any classes with higher misclassification rates.

6.1 Sample Predictions

The model was tested on new fruit images outside the training set and was able to correctly classify most images, with occasional confusion between visually similar fruits (e.g., certain apple and tomato varieties).

7. Challenges Faced

- Limited and imbalanced data for some fruit classes initially.
- Overfitting on the training set, addressed using dropout and data augmentation.
- Long training time on CPU, resolved by using GPU (Google Colab).
- Misclassification between visually similar fruit varieties.

8. Future Scope / Improvements

- Increase the number of fruit classes and dataset size for broader coverage.
- Use transfer learning (MobileNet, ResNet, EfficientNet) for higher accuracy.
- Deploy the model as a web or mobile application for real-time fruit recognition.
- Extend the model to detect fruit ripeness or quality/freshness.
- Optimize the model for edge devices using TensorFlow Lite.

9. Conclusion

This project successfully demonstrates the use of Convolutional Neural Networks for classifying fruit images with good accuracy. Through this internship project, practical experience was gained in data preprocessing, CNN model design, training, evaluation, and the overall deep learning workflow. The project lays a strong foundation for further exploration into more advanced computer vision applications.